MyCodeSchool

User Management în Angular — fără server

9 etape · 3 provocări · datele stau în memorie · Sebastian

Pornire: `npm install` (o singură dată), apoi `npm start` și deschizi `http://localhost:4200`.

Ce construiești: aplicația din `mockup/` (deschide `mockup/index.html` în browser). Nu există server: lista de utilizatori stă într-un serviciu Angular, pornită din `src/app/data/seed-users.ts`. Etichetele `GET /api/...` din mockup le ignori. La refresh, datele revin la cele 5 de început.

Cum lucrezi: etapă cu etapă; fiecare pornește din cea de dinainte. Clasele CSS există deja în `src/styles.css`: copiezi HTML-ul din mockup și înlocuiești datele scrise de mână. Pune tip pe fiecare câmp, parametru și metodă. După fiecare etapă: ce scrie la „Gata când” + un commit.

1. Modelul și serviciul — `models/user.ts`, `services/user.service.ts`

1. În `src/app/models/user.ts`, trei interfețe cu aceleași nume ca în API-ul tău Spring: `User` = `id`, `firstName`, `lastName`, `email` (`string`) și `age: number`, **fără parolă**; `UserCreateRequest` = la fel, fără `id`, plus `password: string`; `UserUpdateRequest` = `firstName`, `lastName`, `email`, `age`.
2. În `seed-users.ts` pui tipul: `export const SEED_USERS: User[] = [...]`.
3. Serviciul îl generezi cu `npx ng g s services/user --type=service --skip-tests`: apare `src/app/services/user.service.ts`, cu clasa `UserService` și `@Service()` deasupra. Angular creează **un singur** obiect din ea, pe care îl primește orice componentă care îl cere.
4. În clasă: `private readonly users = signal<User[]>(SEED_USERS);` și `readonly all = this.users.asReadonly();`

Gata când

`npm run build` trece. Pe ecran nu s-a schimbat nimic încă.

De ce două câmpuri: `users` e privat, deci doar serviciul îl poate modifica (cu `set` / `update`); componentele primesc `all`, din care doar citesc: `all()`.

2. Lista — `users/user-list.ts` + `.html`

1. Componenta `UserList`, selector `app-user-list`, `templateUrl: './user-list.html'`.
2. Primește serviciul cu `private readonly service = inject(UserService);` și expune `protected readonly users = this.service.all;`
3. În `.html` copiezi tabelul din `mockup/index.html` (fără bara de filtre) și pui un singur `<tr>` într-un `@for (user of users(); track user.id)`.
4. Avatarul = prima literă din prenume + prima din nume; identificatorul = primele 8 caractere (`user.id.slice(0, 8)`); badge-ul e `badge-warning` sub 18 ani și `badge-neutral` altfel, cu `[class.badge-warning]="..."`.
5. Blocul `@empty` afișează `empty-state` din `mockup/states.html`: „Niciun utilizator”.
6. În `app.ts`: `imports: [UserList]`; în `app.html`, în `<main>`: `<app-user-list />`.

Pe ecran

```
BA  Bogdan Alexandrescu   41 ani   3f2a91c4   [Editeaza] [Sterge]
RP  Radu Popescu          17 ani   a71b0e33   ...   (badge galben)
AS  Ana Stefanescu        19 ani   c0d84f17
IM  Ioana Marinescu       32 ani   e5b29d60
MD  Mihai Dumitru         24 ani   7d1c4a82
```

3. Statistici — `shared/stat-card.ts` + `.html`

Noțiune nouă: `input()` — date primite de la componenta părinte

1. Componenta `StatCard`, selector `app-stat-card`, cu două intrări: `readonly label = input.required<string>();` și `readonly value = input.required<number>();`
2. În `.html`: un `stat-card` din mockup, cu `{{ label() }}` și `{{ value() }}`.
3. În `UserList`: patru `computed<number>`: `total`, `averageAge` (rotunjită la o zecimală), `minors` (sub 18), `adults`. Pe listă goală, media e `0`.
4. Deasupra tabelului, în `stats-grid`: `<app-stat-card label="Total utilizatori" [value]="total()" />` și încă trei la fel. Nu uita `StatCard` în `imports`.

Pe ecran

```
Total utilizatori 5   Varsta medie 26.6   Minori 1   Adulti 4
```

Indiciu: `label="..."` trimite textul așa cum e; `[value]="total()"` trimite **rezultatul** expresiei. `Math.round(x * 10) / 10` lasă o zecimală.

---

4. Ștergere cu confirmare — `shared/confirm-dialog.ts` + `.html`

Noțiune nouă: `output()` — componenta copil anunță părintele

1. În `UserService`: `remove(id: string): void`, cu `this.users.update((list) => list.filter(...))`.
2. Componenta `ConfirmDialog`, selector `app-confirm-dialog`: intrarea `message` (`string`) și două ieșiri: `readonly confirmed = output<void>();` și `readonly cancelled = output<void>();`
3. În `.html`: `<div class="modal-overlay">` cu fereastra `modal` din `mockup/user-detail.html` (titlu „Stergi utilizatorul?”, mesajul, butoanele „Renunta” și „Sterge definitiv”). Butoanele cheamă `cancelled.emit()` și `confirmed.emit()`.
4. În `UserList`: `toDelete = signal<User | null>(null)`. „Sterge” din tabel doar îl setează; dialogul apare cu `@if (toDelete(); as user)` și îl folosești așa: `(confirmed)="remove(user)"`, `(cancelled)="toDelete.set(null)"`.

Gata când

1. „Renunta” închide dialogul și nu șterge nimic.
2. După ștergerea lui Radu: `Total 4 · Varsta medie 29 · Minori 0 · Adulti 4` — statisticile se refac singure.
3. După ștergerea tuturor: „Niciun utilizator” și `0 · 0 · 0 · 0`.

---

5. Căutare — în `UserList`

1. Deasupra tabelului, `table-toolbar` din mockup, doar cu două câmpuri: căutarea după nume și „Varsta minima” (fără sortare).
2. Două semnale, `search` (`string`) și `minAge` (`number`), legate de câmpuri ca în Demo 5: `#box` + `[value]` + `(input)`.
3. `filtered = computed<User[]>(...)`: numele complet conține textul, fără să conteze literele mari, **și** vârsta e cel puțin `minAge`. Tabelul face `@for` pe `filtered()`; statisticile rămân pe toți.
4. În `@empty` deosebești două stări: nu e niciun utilizator deloc (mesajul de la etapa 2) sau filtrele nu găsesc nimic: „Niciun utilizator nu corespunde filtrelor alese.” + butonul „Sterge filtrele”, care golește ambele câmpuri.

Gata când

```
"nescu"  → Ana Stefanescu, Ioana Marinescu
"POP"    → Radu Popescu
min 30   → Bogdan, Ioana
"radu" + min 30 → mesajul de filtre; „Sterge filtrele” → iar 5 rânduri
```

6. Rutare — `app.routes.ts`

Noțiune nouă: router — o adresă (URL) = o pagină

1. Creezi goale componentele `UserForm` (`users/user-form.ts`, text „Formular”) și `UserDetail` (`users/user-detail.ts`, text „Detaliu”).
2. În `src/app/app.routes.ts`: `export const routes: Routes = [...]` cu: `''` → `redirectTo: 'users'` (+ `pathMatch: 'full'`), `'users'` → `UserList`, `'users/new'` → `UserForm`, `'users/:id'` → `UserDetail`.
3. În `app.config.ts`: `provideRouter(routes, withComponentInputBinding())`.
4. În `app.html`: `<app-user-list />` devine `<router-outlet />`. Linkurile din meniu: `routerLink="/users"` și `routerLink="/users/new"` + `routerLinkActive="active"`, în locul lui `href` și al clasei scrise de mână. Pe „Utilizatori” mai pui `[routerLinkActiveOptions]="{ exact: true }"`, altfel e activ și pe `/users/new`.
5. Butonul „Utilizator nou” din antetul paginii și numele din tabel devin linkuri: `[routerLink]="['/users', user.id]"`.

Gata când

`localhost:4200` sare singur pe `/users`; meniul colorează linkul paginii curente; Back din browser merge; după F5 pe `/users/new` rămâi pe formular.

De ce contează ordinea: scrie `'users/:id'` înainte de `'users/new'` și vezi pe ce pagină ajungi la `/users/new`. Apoi pune-le la loc.

---

7. Pagina de detaliu — `UserDetail`

1. În `UserService`: `findById(id: string): User | undefined`, care citește din `this.users()`.
2. În `UserDetail`: `readonly id = input.required<string>();` — router-ul pune aici `:id` din adresă, datorită lui `withComponentInputBinding()`. Apoi `user = computed(() => this.service.findById(this.id()))`.
3. Pagina după `mockup/user-detail.html`: numele în titlu și grila `detail-grid`.
4. „Sterge” deschide același `ConfirmDialog`; după confirmare, `inject(Router).navigate(['/users'])`.
5. Dacă utilizatorul nu există: `error-panel` cu `404` și „User not found” + link înapoi la listă.

Gata când

Click pe „Ana Stefanescu” → pagina ei; `/users/abc` scris de mână → panoul 404; ștergerea din detaliu te întoarce la o listă cu 4.

---

8. Formularul de creare — `UserForm`

Noțiune nouă: Reactive Forms

1. În `UserForm`: `imports: [ReactiveFormsModule]` și `protected readonly form = inject(FormBuilder).nonNullable.group({ ... })`, cu câte un control pe câmp. Fiecare control: `['valoare de start', [reguli]]`, de ex. `email: ['', [Validators.required, Validators.email]]`.
2. Regulile: prenume și nume obligatorii; email obligatoriu și valid; parola minim 8 caractere (`Validators.minLength(8)`); vârsta între 1 și 120 (`Validators.min`, `Validators.max`).
3. `.html` după primul card din `mockup/user-form.html`: `[formGroup]="form"` pe container și `formControlName="email"` pe fiecare input.
4. Sub fiecare câmp, mesajul din mockup apare doar dacă `form.controls.email.invalid && form.controls.email.touched`; în aceeași condiție, inputul primește `is-invalid`. „Salveaza” e `[disabled]="form.invalid"`.
5. În `UserService`: `add(data: UserCreateRequest): User` construiește un `User` nou cu `id: crypto.randomUUID()` și câmpurile din `data`, **fără parolă** — exact ce face și serverul: primește parola, dar n-o trimite niciodată înapoi. Plus `emailExists(email: string): boolean`.
6. La salvare: dacă emailul există, `error-panel` cu `409` și „Email already used by another user”; altfel `add(this.form.getRawValue())` și navighezi la `/users`.

Gata când

1. Nicio eroare roșie la deschidere; apar doar după ce intri și ieși dintr-un câmp.
2. Exemplele greșite din mockup (nume gol, `cristian.tudor`, `1234`, vârsta `0`) dau mesajele din mockup (la vârstă: „Age must be between 1 and 120”), iar butonul e inactiv.
3. `radu.popescu@gmail.com` → panoul 409; un email nou → 6 utilizatori, media recalculată.

9. Editare — același `UserForm`, ruta `users/:id/edit`

1. Ruta nouă `'users/:id/edit'` → `UserForm`; „Editeaza” din tabel și din detaliu duc acolo.
2. În `UserForm`: `readonly id = input<string>();` (fără `required`: la creare nu există). În `ngOnInit()` (clasa `implements OnInit`), dacă există `id()`: completezi formularul cu `this.form.patchValue(user)` și oprești parola cu `this.form.controls.password.disable()`. Un control oprit nu se mai verifică.
3. Titlul („Utilizator nou” / „Editare utilizator”) și câmpul parolă depind de `id()`.
4. În `UserService`: `update(id: string, data: UserUpdateRequest): void` — parola nu face parte din editare. `emailExists` primește și id-ul de sărit, ca să nu te blocheze propriul email.

Gata când

Ioana → vârsta 17 → Salveaza: badge galben, `Minori 2`, media `23.6`. Editare fără nicio schimbare → se salvează. Emailul lui Radu pus la Ioana → 409.

De ce în `ngOnInit`, nu în constructor: pune un `console.log(this.id())` în ambele și compară.

Provocări — fără indicii

P1. Sortare

Select-ul de sortare din mockup, cu cele 4 variante. Sortarea se aplică după filtre; lista din serviciu nu se schimbă.

P2. Paginare

Câte 3 utilizatori pe pagină, cu bara `pagination` din `mockup/index.html`: „Afișez 1–3 din 5”, butoanele de pagină, Înapoi/Înainte inactive la capete. O căutare nouă te duce pe pagina 1; dacă ștergi ultimul utilizator de pe ultima pagină, nu rămâi pe o pagină goală.

P3. Datele rămân după refresh

Fără server: lista se salvează în `localStorage` la fiecare schimbare și se citește de acolo la pornire; doar dacă acolo nu e nimic pornești din `SEED_USERS`. Un buton „Reseteaza datele” readuce cei 5 de la început. Toată logica stă în `UserService`; componentele nu se schimbă.